
计算机系统基础回顾

许向阳

xuxy@hust.edu.cn

不知道自己不知道



知道自己不知道



不知道自己知道



知道自己知道

课程内容

L01 宏论计算机系统基础

L02 计算机工作的基本过程

L03 Intel 中央处理器

L04_05 主存储器及数据在计算机内的表示形式

L06 寻址方式

L07 常用机器指令

L08 分支程序

L09 循环程序

L10_12 函数调用

L13 串操作指令

L14 SIMD 指令

L15_16 程序的链接

L17_19 异常控制流

L20 华为智能基座_华为鲲鹏处理器简介

虚拟内存管理

计算机系统基础回顾

计算机工作的基本原理

数据的机器级表示与处理

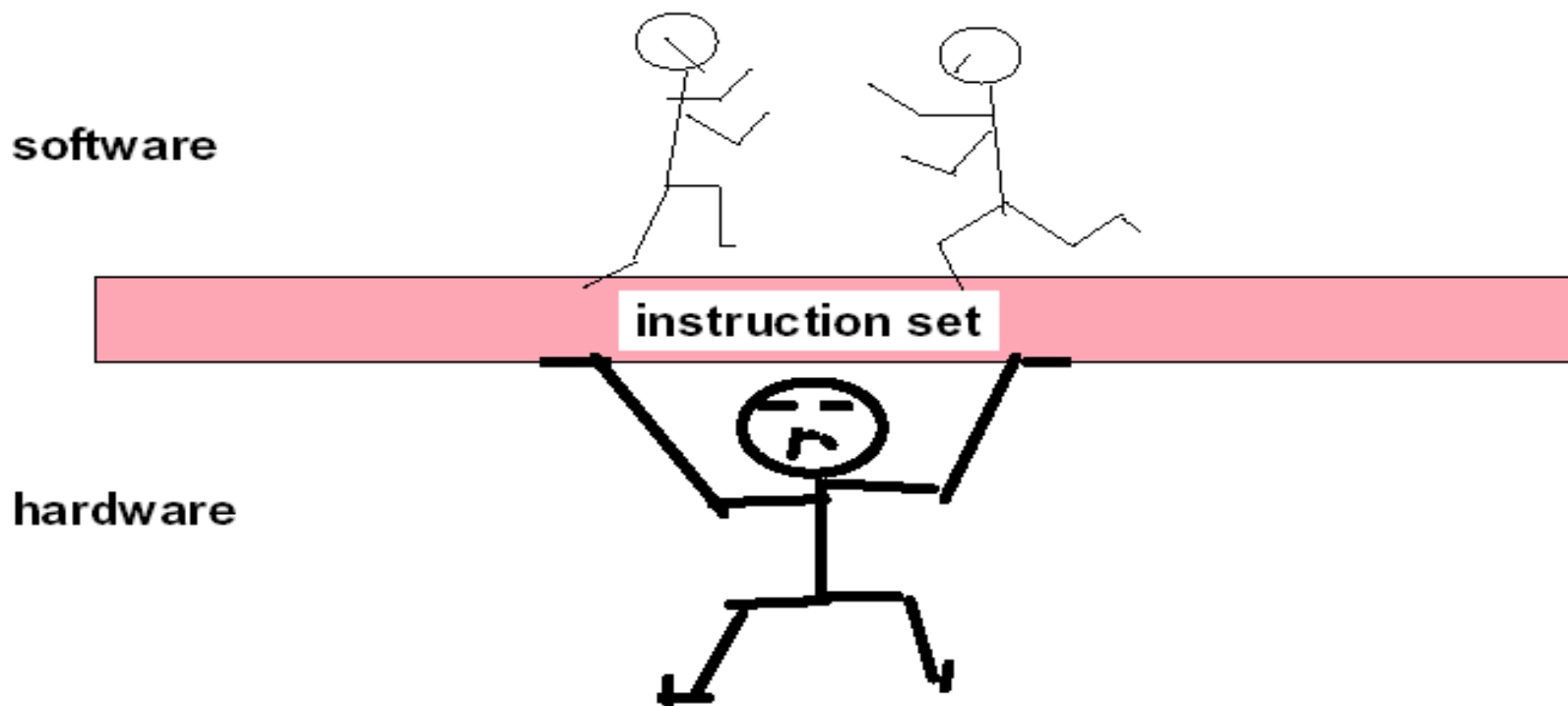
程序的机器级表示

程序的链接

程序的运行

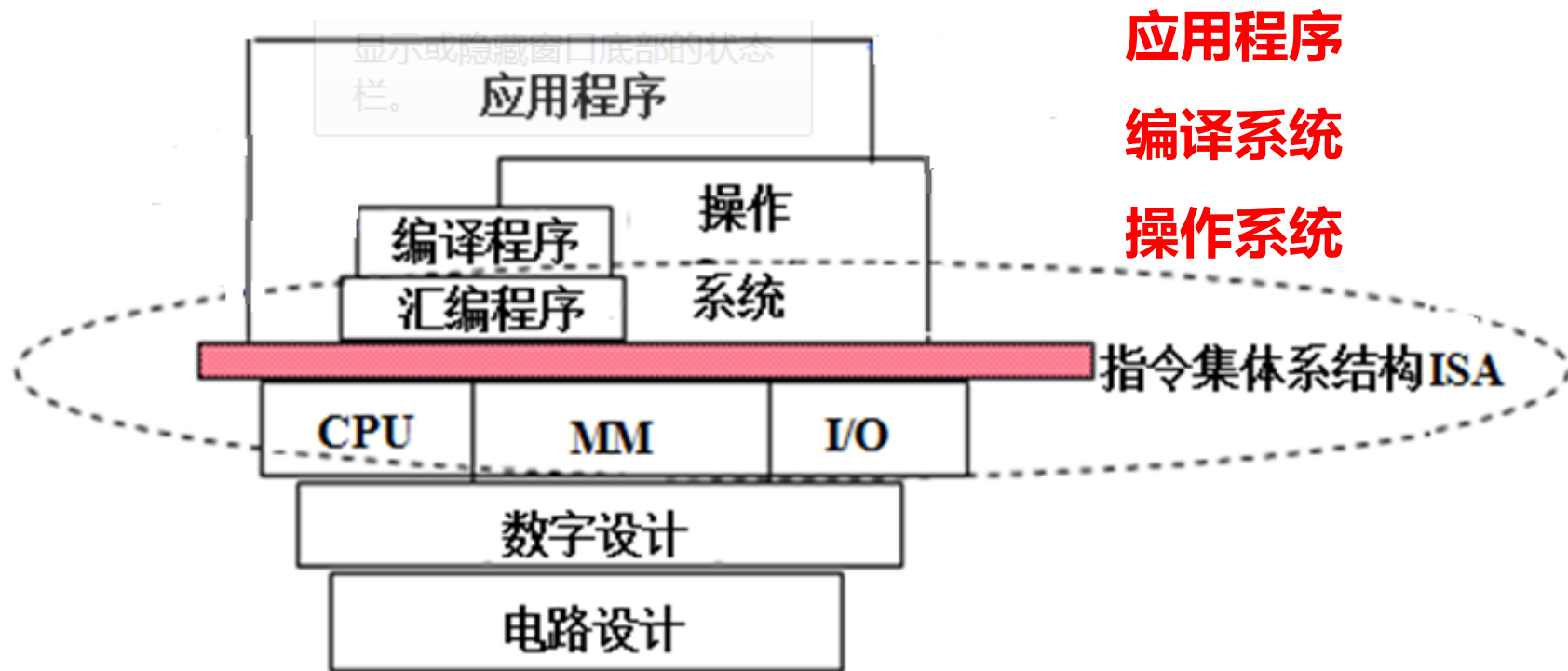
计算机系统基础回顾

若干部分组成，相互联系、相互作用，形成具有某些功能的整体



指令集体系结构 Instruction Set Architecture

计算机系统基础回顾



ISA是对硬件的抽象

所有软件功能都建立在ISA之上

计算机系统基础回顾

应用程序二进制接口 (Application Binary Interface, ABI)

应用的组成部分之间、一个应用和它的库之间、应用程序和操作系统之间的 **接口**。

过程间调用约定 (参数和返回值传递等)

系统调用约定 (系统调用的参数、调用号如何传递,
如何从用户态陷入操作系统内核等)

目标文件的二进制格式

函数库使用约定

寄存器使用规定

程序的虚拟地址空间划分

计算机系统基础回顾

应用程序接口(Application Program Interface, API)

操作系统留给应用程序的一个调用接口，应用程序通过调用操作系统的 API 而使操作系统去执行应用程序的命令

Windows API

Linux API

ISA：软件与硬件之间最底层接口

ABI：是二进制代码级的接口

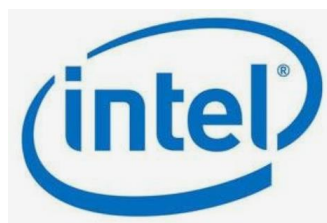
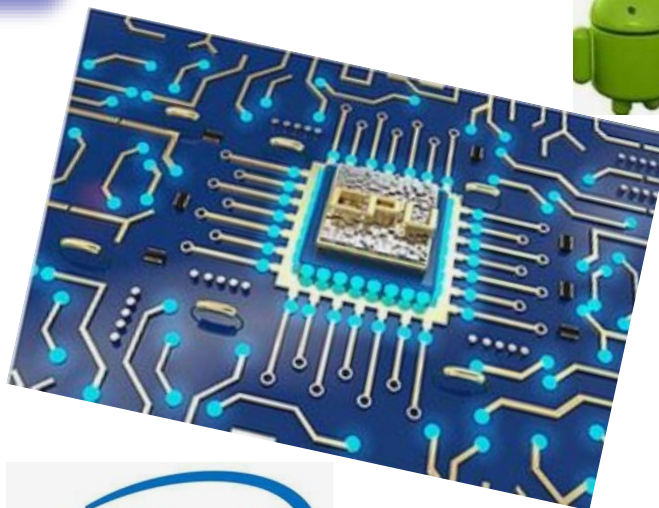
API：是源代码级的接口



接口

计算机系统基础回顾

CISC、RISC



指令集体系结构 Instruction Set Architecture

计算机系统基础回顾



计算机系统基础回顾



简单 or 复杂？

呆板 or 灵活？

表象 or 本质？



计算机系统基础回顾

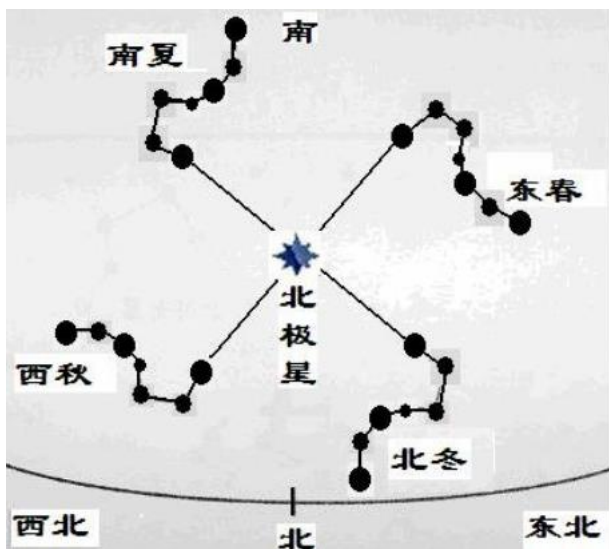


天罡北斗阵



知乎 @胡超御定奇门宝鉴

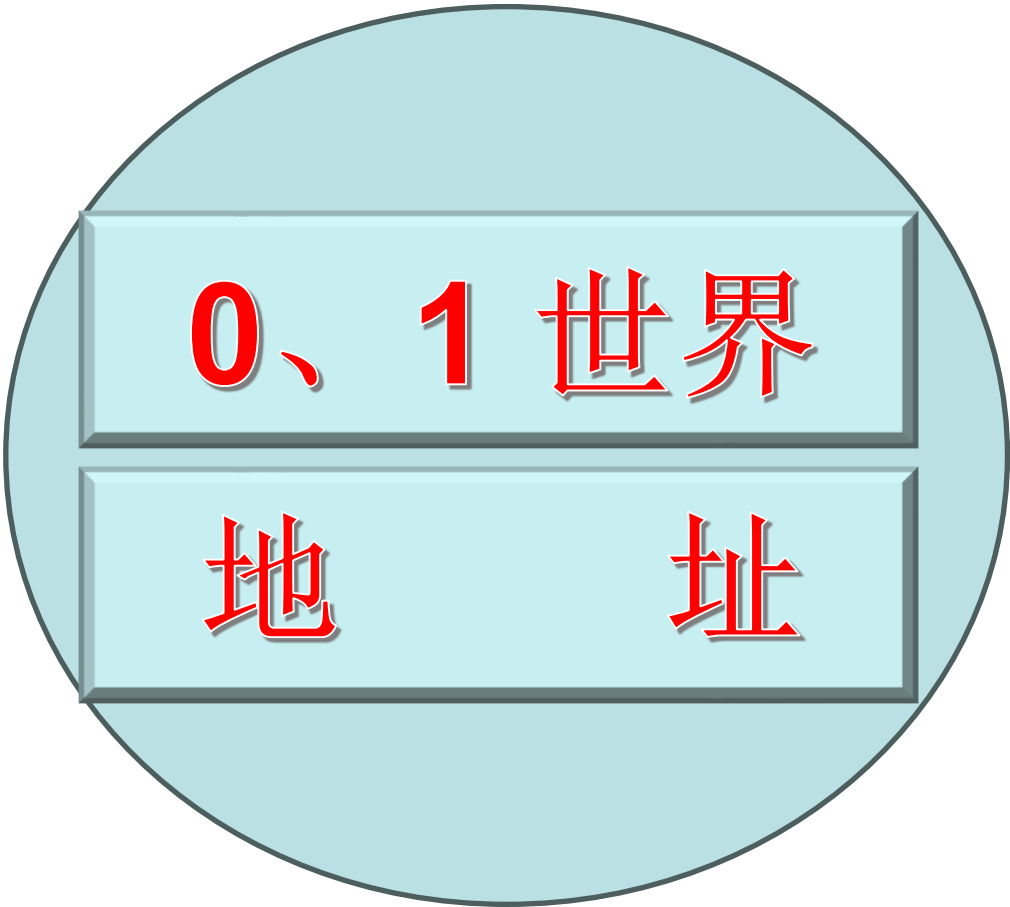
计算机系统基础回顾



斗转星移

亘古不变

计算机系统基础回顾

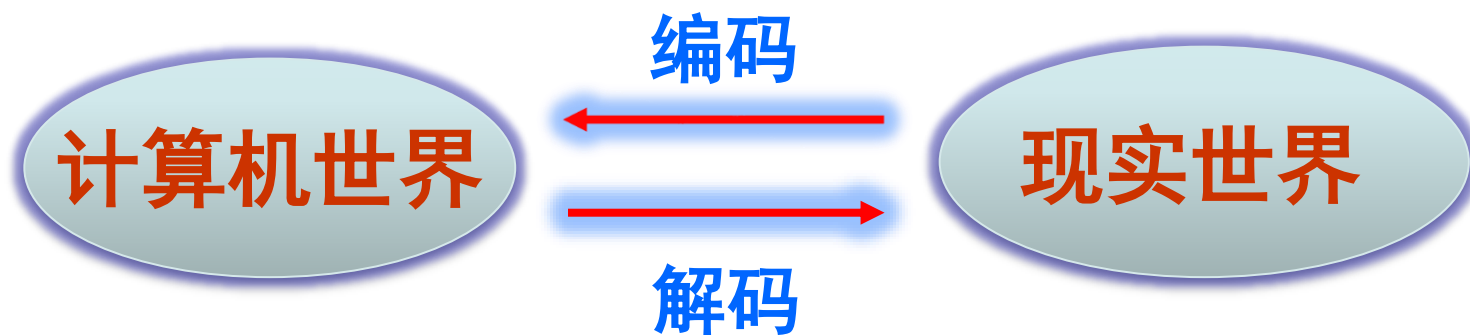


0、1 世界

地 址

计算机系统基础回顾

计算机是 0-1 的世界



ASCII、Unicode、UTF8、UTF16

有/无符号整数编码

IEEE754 浮点数编码

机器指令编码格式 Intel、ARM.....

目标文件格式 ELF、PE、.....

文字、图像、视频、声音、表格、网页.....

编码
规则
标准

计算机系统基础回顾

计算机是 0-1 的世界

0x31
0x32
0x33
0x34
0x35
0x36
0x37
0x38
0x00

机器指令 ?

```
xor dword ptr [edx], esi    31 32
xor esi, [esi + 383736h]    33 34 35 36 37 38 00
```

字符串 ? "12345678", 0

整型(int) ? (有符号整型 or 无符号整型)

875770417 943142453

单精度浮点数 (float) ?

0.000000 0.000044

双精度浮点数 (double) ?

0.000000

地址 ?

0x34333231 0x38373635

其他的结构 ? 其他的数据 ? ...

二进制数据本身是“沉默的”，它的意义完全由上下文赋予。

没有上下文，二进制只是一堆无意义的比特。

计算机系统基础回顾

存储程序的基本工作原理



- 任何要计算机完成的工作都要先被编写成程序;
- 程序和原始数据要存入主存并启动执行;
- 一旦程序被启动, 计算机不需操作人员干预;
- 自动完成逐条取出指令和执行指令的任务。

计算机系统基础回顾

自动、逐条 取出指令和执行指令



指令在哪？

如何自动？

RIP/EIP：指令指示器、程序计数器（PC）

□ **取指令**

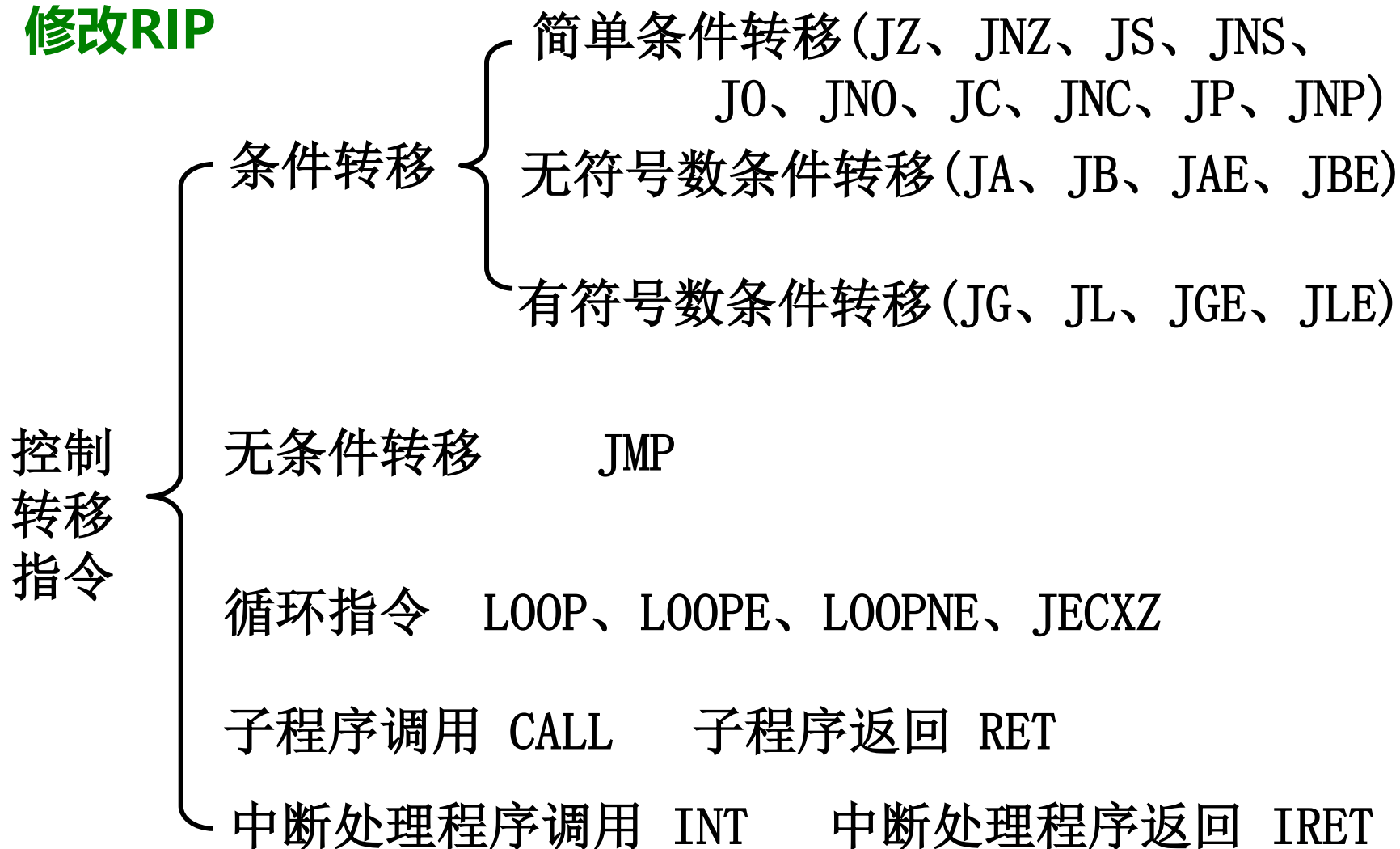
□ **指令译码：RIP 自动加上指令的长度**

□ **指令执行：与RIP 无关的指令（数据传送、算术运算...）**

与RIP有关的指令（控制转移）

计算机系统基础回顾

修改RIP



计算机系统基础回顾

修改RIP 标号、函数名、变量名 都是 地址的符号表示

RIP 变化方式及应用

{	直接转移	JMP	标号
		CALL	子程序/函数
{	间接转移	JMP	OPD (除立即数外)
		CALL	OPD
{	相对转移	(RIP) + 位移量 $\rightarrow$ RIP	
	绝对转移	(OPD) $\rightarrow$ RIP	
{	段内转移	(CS) 不变	
	段间转移	(CS) 改变	

地址表法

函数指针

函数指针数组

计算机系统基础回顾

正常程序流下的 **RIP** 变化

中断/异常导致的 **RIP** 变化



中断、异常？

中断、异常
处理程序设计

计算机系统基础回顾



指令在哪? RIP

如何自动? RIP的变化

指令中含有什么信息?

操作 与 操作对象的地址

计算机系统基础回顾

操作码：进行的操作

**数据传送、带条件的数据传送、算术运算、
逻辑运算、移位运算、位操作和字节操作、
标志位控制、控制转移、串操作、I/O 指令
成组运算、饱和运算、MMX/SSE/AVX指令**

计算机系统基础回顾

操作对象 → 操作对象的地址码

寻址方式

Intel格式

At&t 格式

立即寻址

n

$\$n$

寄存器寻址

R

$\%R$

直接寻址

$[v]$

v

寄存器间接寻址

$[R]$

$(\%R)$

变址寻址

$v[R * F]$

$v(, \%R, F)$

基址加变址寻址

$v[BR + IR * F]$

$v(\%BR, \%IR, F)$

相对寻址 $v(\%RIP)$: 全局变量的地址、只读串的地址等

虚拟地址、逻辑地址
物理地址是怎么形成的？

OS 内存管理：分页、分段、段页、扁平

CPU: MMU

计算机系统基础回顾

操作对象

立即数 Immediate Operands

寄存器操作数 Register Operands

内存操作数 Memory Operands

IO操作数 外部设备寄存器

计算机系统基础回顾

有几个操作对象？ 操作的结果又放在哪？

➤ 源操作与目的操作数的摆放顺序差异

AT&T 格式中，目的操作数在源操作数的右边；

Intel 格式中，目的操作数在左边

➤ 寻址方式表达差异

AT&T : 段寄存器:位移量(基址寄存器, 变址寄存器, 比例因子)

Intel : 段寄存器:[基址寄存器+变址寄存器*比例因子+位移量]

➤ 寄存器表示方式上的差异 ‘%’ 作为前缀

➤ 立即数表示的差异

用 ‘\$’ 前缀表示一个立即操作数, 用 0x表示16进制

\$0x0a 0ah

➤ 特别注意：立即操作数 与 地址偏移量是两个不同的概念

计算机系统基础回顾

AT&T 格式

`movl -4(%ebp), %eax`

功能: $([ebp]-4) \rightarrow eax$

`movl array(, %eax, 4), %eax`

功能: $(array[eax*4]) \rightarrow eax$

`movw array(%ebx, %eax, 4), %cx`

`movb $4, %fs:(%eax)`

`movl 0x40001200, %eax`

Intel 格式

`mov eax, [ebp - 4]`

`mov eax, [eax*4 + array]`

`mov cx, [ebx + 4*eax + array]`

`mov fs:[eax], 4`

`mov %eax, ds:[40001200h]`

数据的存放

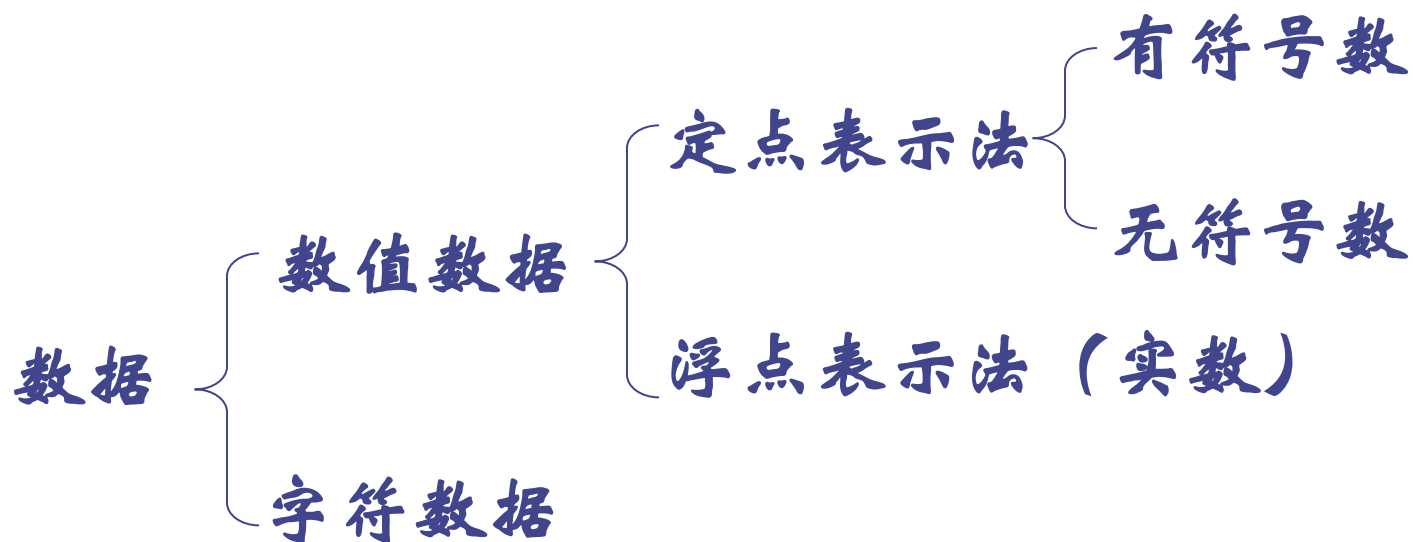
Intel 小端 (Little-Endian) 存放:

若一个数据占多个字节，这些字节连续存放，
数据的低字节存放在地址最小的单元中，
数据的高字节存放在地址最大的单元中，
数据的地址是最低字节的地址（即最小的地址）。

从 CPU 的角度看，基本的访问单位是 字节、字、双字、四字；最小的访问单位是字节。

数据指的是：字节、字、双字、四字型的数据

数据的表示



数据类型转换： 同一个数的不同表现形式

整型值之间的转换： (1) 等长直接拷贝；
(含有/无符号数) (2) 超长截断；
(3) 保持右边的类型，扩展。

地址类型转换： 相同的内容，解读成不同类型的数。

数据的表示

一维数组中，各个元素存储的位置关系

二维/多维数组中，各个元素存储的位置关系

结构变量中各个字段的存储关系

联合(union)中个字段的的关系

结构数组中各元素、各字段的存储关系

指针变量中的存储内容

数据的访问

C语言中各种访问变量表达形式

与机器语言中访问形式的对应关系

C语言中各种变量的空间分配位置

全局变量

局部变量（非静态，auto, 自动型）

静态局部变量

C语言中各种类型的变量可以对应哪一种或者哪几种寻址方式？

数据传送指令

1、一般数据传送指令

MOV、MOVSX、MOVZX

2、堆栈操作指令

PUSH、POP

3、地址传送指令

LEA

算术运算指令

1、加法指令

INC、ADD、ADC

2、减法指令

DEC、NEG、SUB、SBB、CMP

3、乘法指令

IMUL、MUL

4、除法指令

IDIV、DIV

注意对标志位的影响： CF、ZF、OF、SF

逻辑运算指令

求反	NOT	按位取反	~
逻辑乘	AND	按位与	&
测试指令	TEST		
逻辑加	OR	按位或	
按位加	XOR	按位异或	^

逻辑非 ! VS 按位反 ~

逻辑与 && VS 按位 &

逻辑或 || VS 按位或 |

注意对标志位的影响: ZF、SF

移位指令

(1) 算术左移 **SAL** **S**hift **A**rithmetic **L**eft

(2) 逻辑左移 **SHL** **S**hift Logical **L**eft

(3) 逻辑右移 **SHR** **S**hift Logical **R**ight

(4) 算术右移 **SAR** **S**hift **A**rithmetic **R**ight

转移指令

- 简单条件转移
JZ、JNZ、JS、JNS、JC、JNC
- 无符号数条件转移
JA、JB、JAE、JBE
- 有符号数条件转移
JG、JL、JGE、JLE
- 无条件转移 JMP
- 循环指令 LOOP
- 子程序调用 CALL
子程序返回 RET
- 中断处理程序调用 INT
中断处理程序返回 IRET

串操作指令

SIMD 指令

成组运算

饱和运算 与 环绕运算

更快的程序

由编译器优化生成运行速度更快的程序？

自己编写运行速度更快的程序？

编译器敢于对所有程序段优化吗？

更快的程序

充分利用 CPU 提供的功能

- 指令流水线
 - ❑ 分支向无分支转化
 - ❑ 循环展开
 - ❑ 指令调整顺序
- Cache
 - ❑ 列序转行序访问
- 指令执行速度
 - ❑ 移位代替乘法
 - ❑ 加法代替乘法
 - ❑ 串操作指令
 - ❑ SIMD指令
- 存储访问
 - ❑ 变量与寄存器绑定

更快的程序

减少执行的代码 （与CPU 无关）

- 能在编译时，产生结果的语句或函数被代换
- 没有实际执行意义的语句被抛弃
- 函数替换，减少参数传递，函数调用与返回

更鲁棒的程序

利用 编译器自动生成的代码，增加安全性

- 缓冲区溢出攻击的防范
- 数组越界的防范
- 读取未初始化变量的值的防范
- 栈的平衡性检查
- 随意跳转的防范

更鲁棒的程序，编译器能够代劳吗？

更鲁棒的程序

在写程序时，注意计算机世界的数是有限长度；
与纯数学领域的计算有所不同！

- 一个非零数 x ，可以有 $x \neq -x$ ；
- $x * x$ 不一定大于等于 0；
- $x < y$ ； 与 $x - y < 0$ 不等价；
- 有符号数、无符号数的操作

C 程序的编译、链接

- 变量的存储内容的表示
- 变量的地址、存储空间的分配与回收
- 语句与指令段的对应关系
 - ❑ 代码节、数据节、只读数据节、BSS节
- 可重定位目标文件的结构
 - ❑ 代码节的重定位节
 - ❑ 数据节的重定位节
 - ❑ 符号表节
 - ❑ 字符串节
- 链接
 - ❑ 符号：全局变量名、非静态局部变量名、函数名
 - ❑ 符号解析：建立符号定义与符号引用之间的关联关系
 - ❑ 重定位
 - ❑ 定位方式
- 加载时动态库的链接

异常控制流

- 异常控制流
- 中断、异常（故障、陷阱、中止）
- 中断异常的处理过程
- `set jmp` `long jmp` 运行环境的切换
- 异常处理链 进程控制块

眼望星空、勇攀高峰





祝愿同学们前程似锦，明天更好！